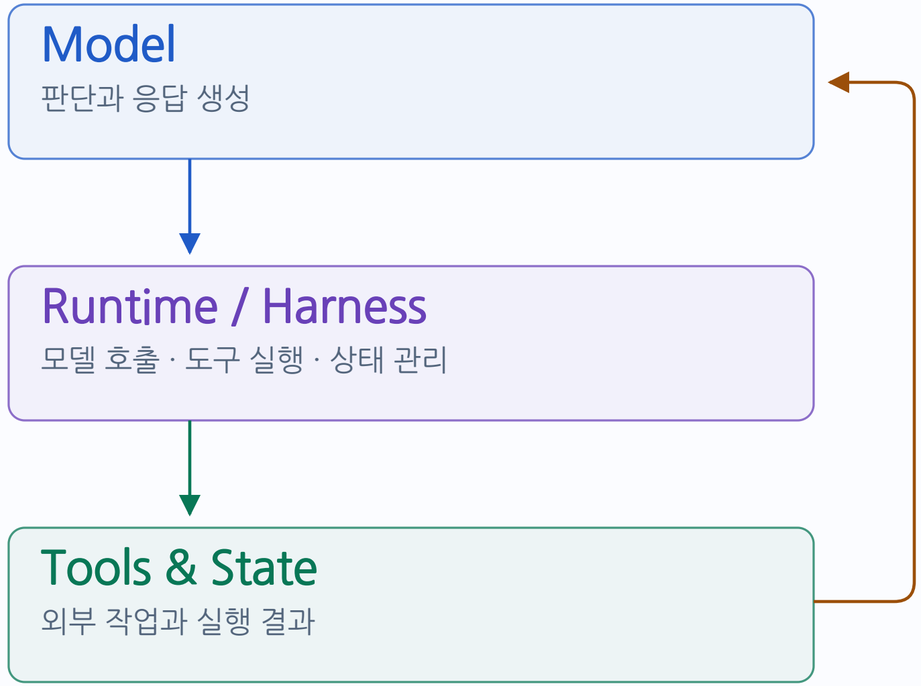
Agent Framework

세부 구조를 살펴봅니다

2차 세미나

Hermes Claude Code PydanticAI

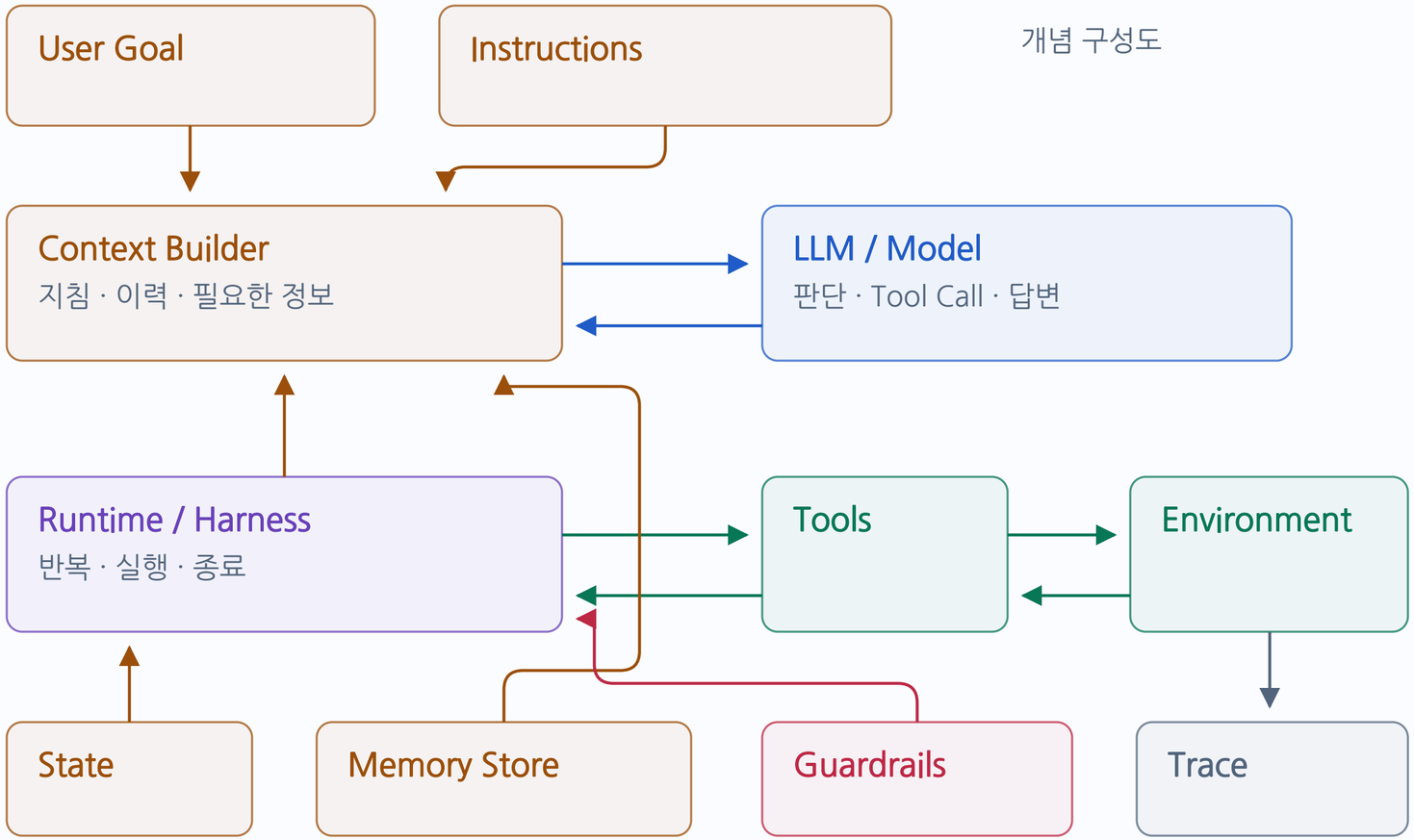


Agent Framework의 구성

목표가 모델 호출과 도구 실행으로 이어지는 과정

Agent를 이루는 주요 구성요소

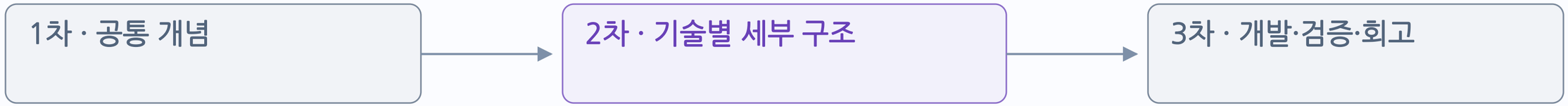
목표와 지침을 읽고
모델과 도구를 연결해
작업을 이어갑니다.



각 기술이 이 구조를 어떻게 구현하는지 살펴봅니다.

2차 세미나 진행 계획

1차 세미나의 내용을 기반으로 실제 기술별 세부 사항을 살펴봅니다.



모델 개발·서빙

AI Studio · vLLM · SGLang

강준혁

Agent 개발·워크플로

DeepAgents · LangChain · LangGraph

박건

Agent Harness·SDK

Hermes · PydanticAI · OpenAI · Claude

남우성

> 3차에서는 직접 개발하고, 결과를 검증하며 회고합니다.

Agent Harness와 SDK

완성된 Agent를 확장하거나, 코드로 필요한 기능을 구성합니다.

Agent Harness

갖춰진 실행환경에서 시작

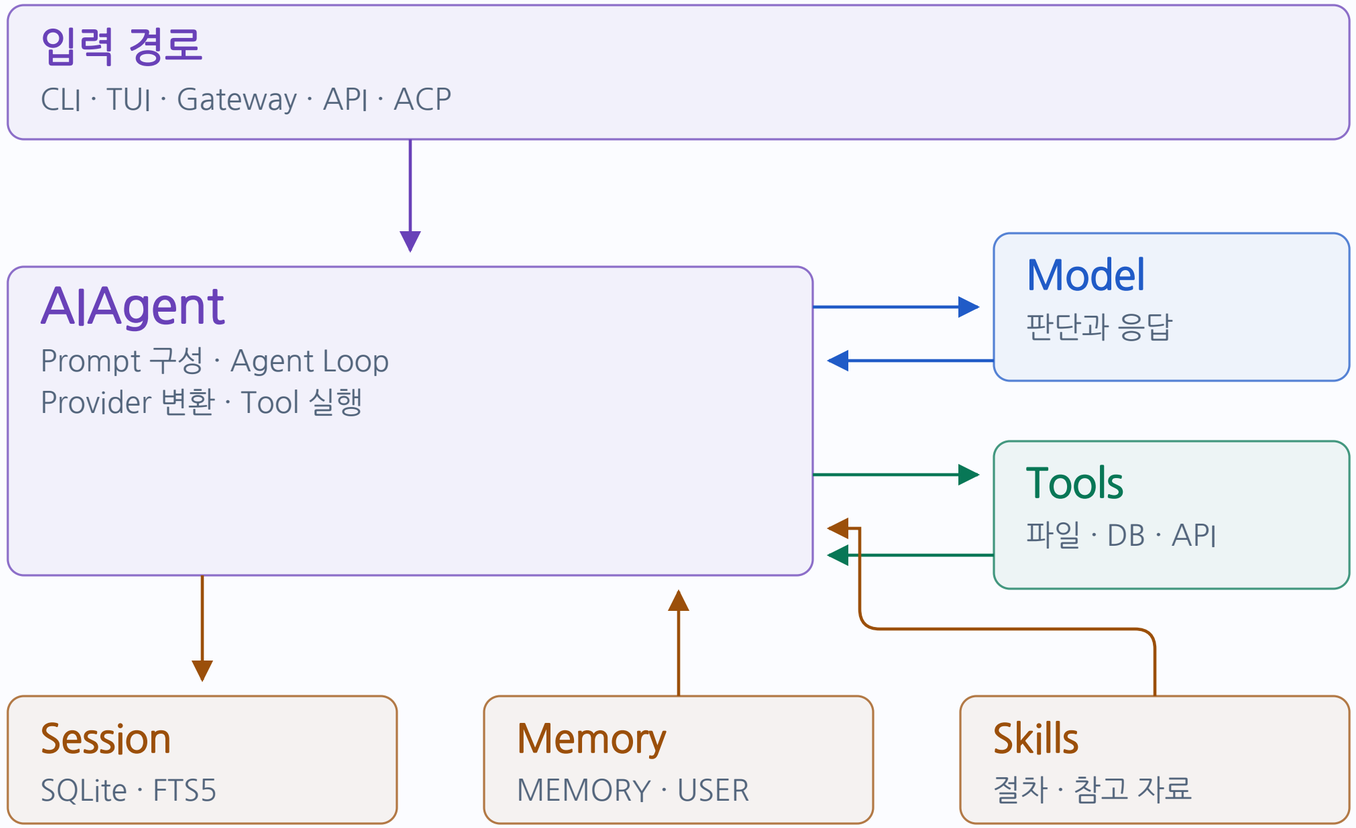
- **Hermes** — 범용 Agent Harness
- **Claude Code** — 코딩 작업을 위한 Harness

Agent SDK

코드로 Agent 구성

- **PydanticAI** — Pydantic 기반 Agent SDK
- **OpenAI Agents SDK**
- **Claude Agent SDK**
- **Google ADK**
- **Microsoft Agent Framework**

> Hermes와 PydanticAI를 중심으로 살펴봅니다.



Hermes

범용 Agent Harness

모델과 도구를 연결하고
대화와 작업을 이어가는
실행환경

전체 구조를 먼저 보고, 각 기능의 동작을 살펴봅니다.

Hermes는 범용 Agent Harness입니다

모델 호출부터 도구 실행과 상태 관리까지 기본 실행환경을 제공합니다.

실행 루프

Agent Loop · Tool Calling

취소·중단 · 대체 모델 처리

입력과 출력

CLI · TUI · Desktop

메신저 Gateway · HTTP API · ACP

도구와 확장

터미널·파일·웹·브라우저·코드 실행

Toolsets · MCP · Plugins

기억과 검색

SQLite 세션 · FTS5 검색

Memory 파일 · 외부 Memory Plugin

작업 절차

작업별 Skills · Agent 작성 Skills

Curator로 사용 이력 관리

위임과 운영

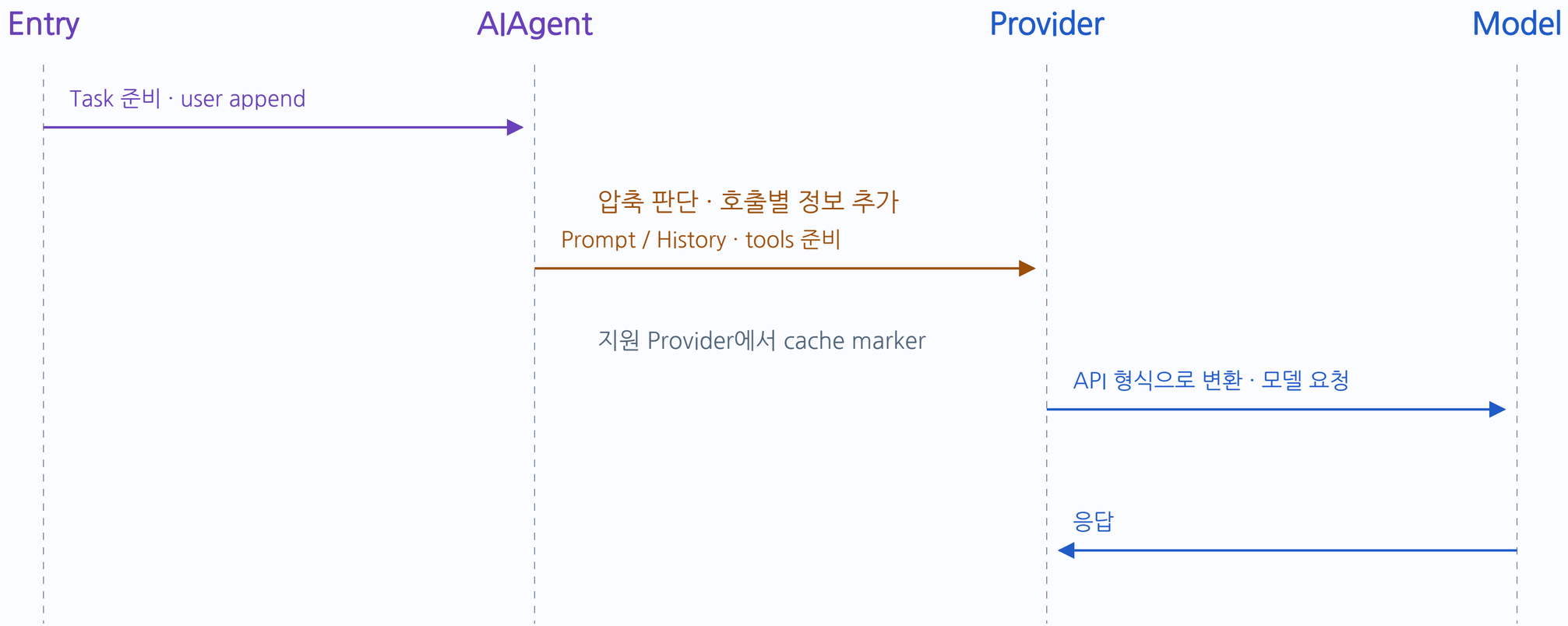
Subagent · Cron · Hooks

승인·격리 · 파일 복구

실제 사용 가능한 기능은 설치 환경과 설정에 따라 달라집니다.

사용자 입력에서 모델 요청까지

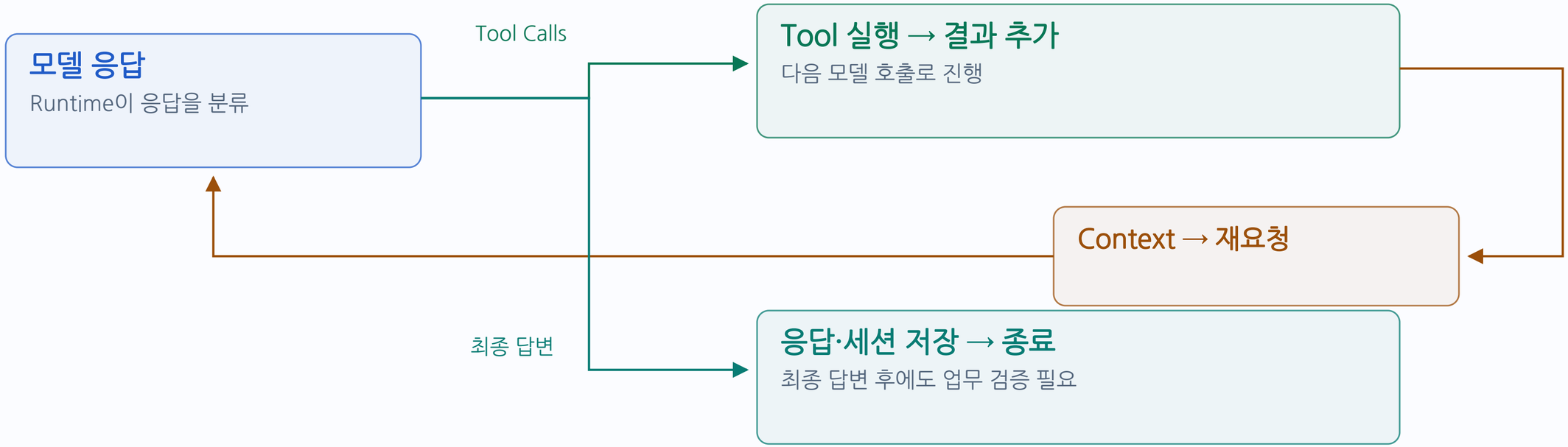
한 Turn을 시작할 때 준비하는 정보



최종 요청에는 지침, 대화 이력, 도구 정의가 함께 담깁니다.

Tool을 실행할 때와 답변을 마칠 때

모델 응답에 따라 반복하거나 종료합니다.



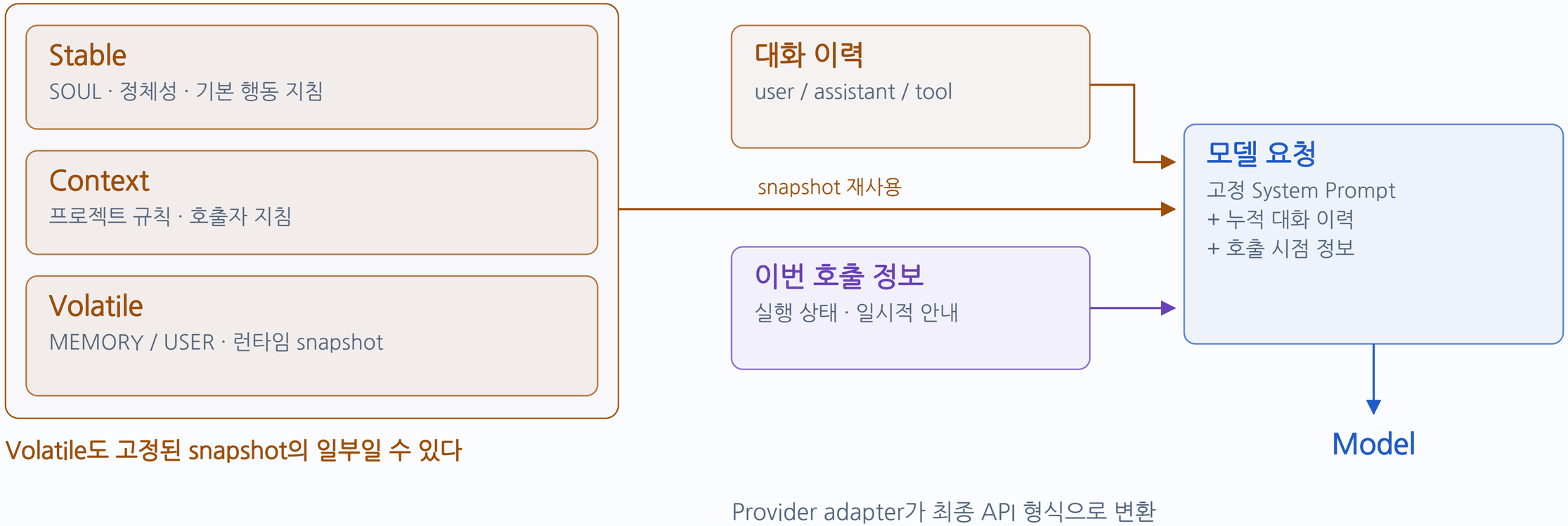
오류·중단·예산 Tool 오류 → 결과 메시지 / API 오류 → retry·fallback / 중단·예산 → 종료 경로

> Tool 오류와 모델 API 오류는 복구 방법이 다릅니다.

고정된 Prompt와 매번 달라지는 정보

세션의 지침에 대화 이력과 호출 시점 정보를 더합니다.

세션에서 재사용하는 System Prompt



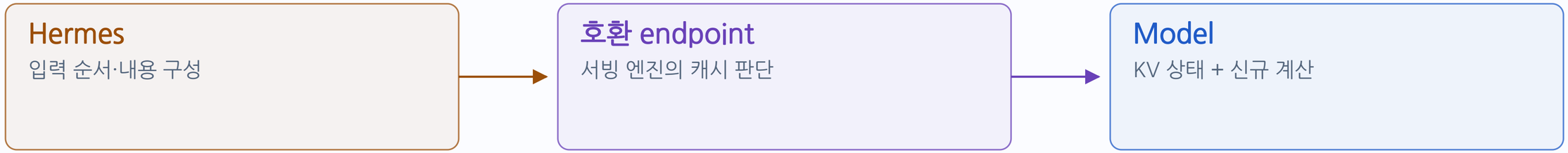
Volatile 계층도 세션 안에서는 같은 snapshot을 사용할 수 있습니다.

같은 Prefix의 계산을 재사용하기

반복되는 입력 앞부분은 추론 서비스가 캐시로 처리할 수 있습니다.



총 10,000 tokens · 설명용 수치



입력은 다시 보내지만, 일부 계산은 재사용할 수 있습니다.

긴 대화에서 남길 것과 줄일 것

과거 내용을 줄이면서 최근 문맥과 Tool 호출 관계를 보존합니다.

Before

누적된 대화·Tool 결과

오래된 대화

이미 처리한 결정·관찰

Tool Call + Tool Result

대응 ID가 있는 실행 기록

최근 대화

현재 요청·미해결 항목

After

요약 + 최근 문맥 + Tool 쌍

요약과 보존 근거

중요 사실·결정·원본 경로

Call / Result 쌍 유지

호출 ID와 결과를 함께 보존

최근 문맥 유지

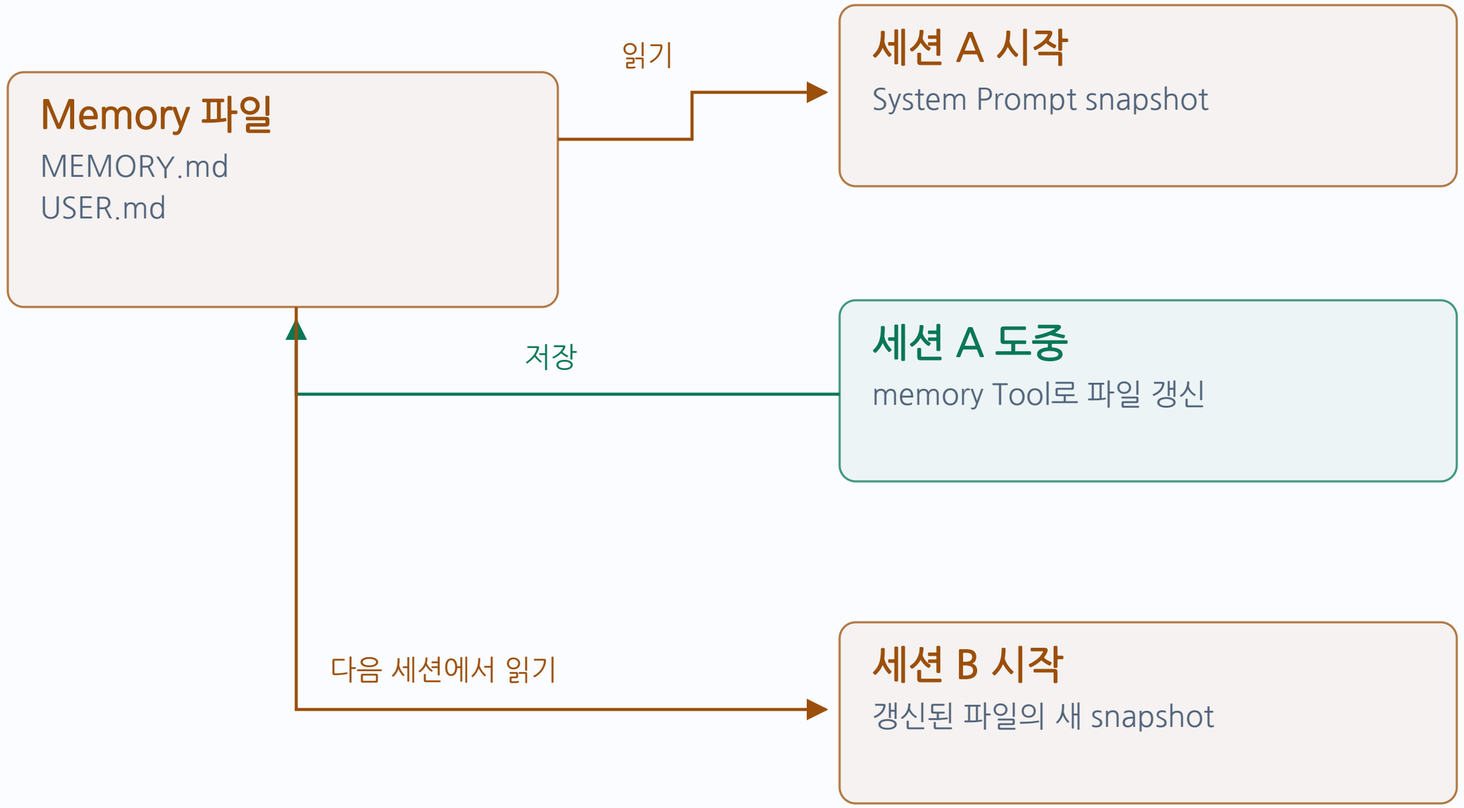
현재 작업을 이어갈 정보

발동 위치: Tool loop 안 / Gateway의 Turn 전 위생 처리

> 압축 후에도 호출과 결과가 서로 대응해야 합니다.

저장한 Memory는 언제 반영될까?

파일을 바꾸는 시점과 세션이 읽는 시점은 다릅니다.



현재 system snapshot은
자동 교체되지 않음

> 현재 세션의 system snapshot은 파일 저장만으로 바뀌지 않습니다.

기억할 사실과 다시 찾을 대화

Memory, Session History, 외부 Provider의 역할

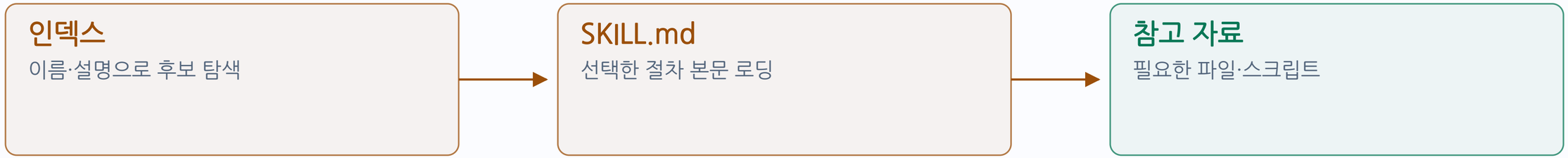
정보 계층	저장·접근	모델 입력으로 들어오는 방식
Built-in Memory	MEMORY.md / USER.md	세션 시작 snapshot
Session History	state.db · SQLite / FTS5	session_search로 선택한 기록
External Provider	별도 저장소·Plugin	확장 구현의 조회·전달

정확한 장비 ID와 유사 사례 질문으로 검색 결과를 비교합니다.

> 과거 대화는 필요한 기록을 검색해 현재 Context에 넣습니다.

Skill을 읽고 다시 사용하는 과정

목록에서 절차를 고르고, 필요한 자료를 불러옵니다.



문서형 절차 패키지

```
wafer-anomaly-review/
SKILL.md
references/
scripts/
```

유지 관리

skill_manage · 생성·수정·삭제

Curator

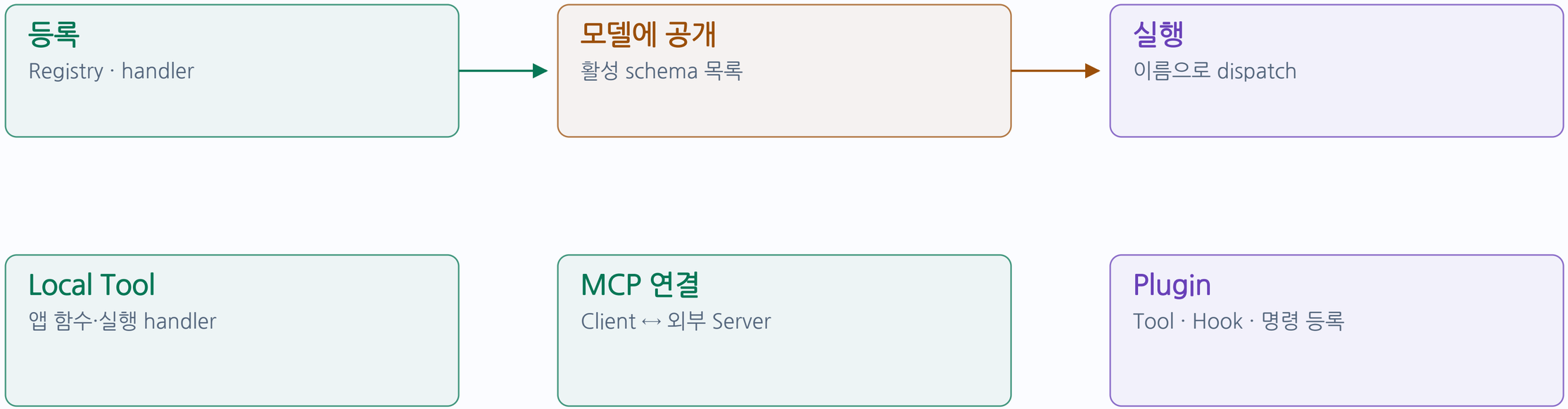
active → stale → archived

Agent 생성 Skill의 사용 이력 기반

저장한 절차를 다시 쓰기 전에 근거와 버전을 확인합니다.

Tool 등록과 MCP·Plugin 연결

실행 함수, 외부 서버, 런타임 확장을 연결하는 방법



memory / delegate_task 등 Agent 상태 도구에는 별도 처리 경로가 있을 수 있다.

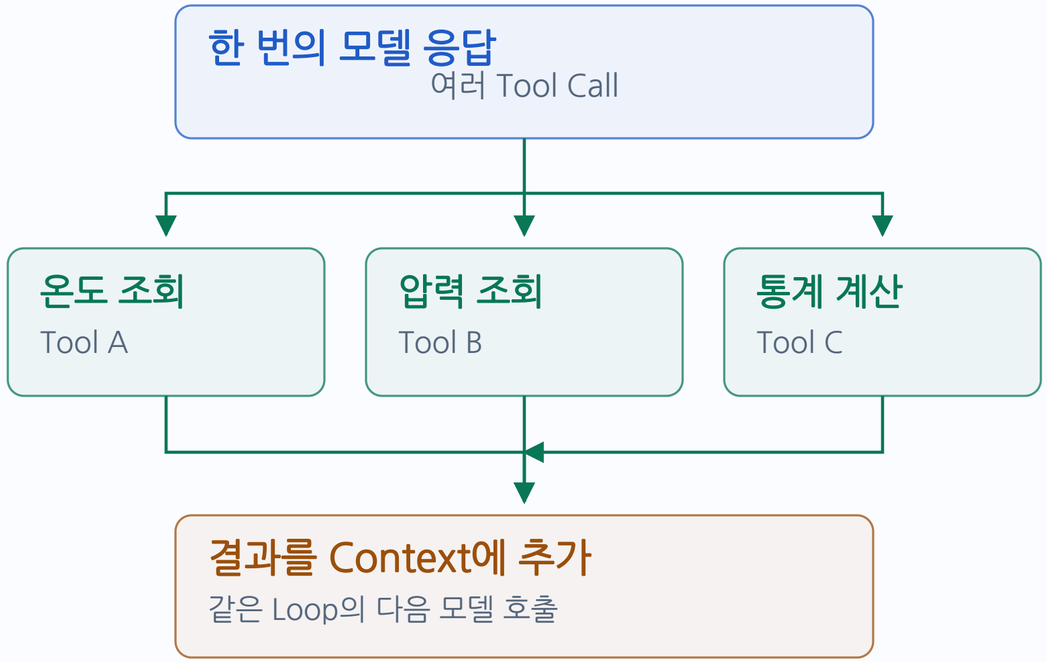
> 도구에 따라 실행 경로와 관찰 가능한 지점이 달라집니다.

Tool 병렬 실행과 Subagent 위임

함수를 함께 실행하는 경우와 별도 Agent에게 맡기는 경우

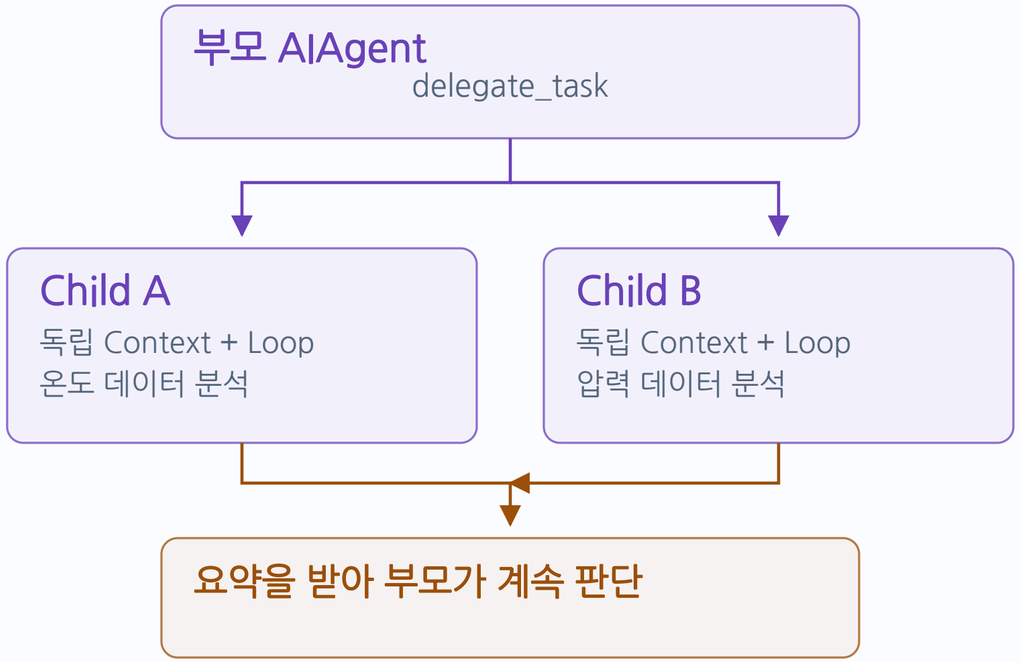
여러 Tool

한 Loop가 요청하고 결과를 모은다



여러 Child Agent

각자 모델 호출과 Context를 가진다

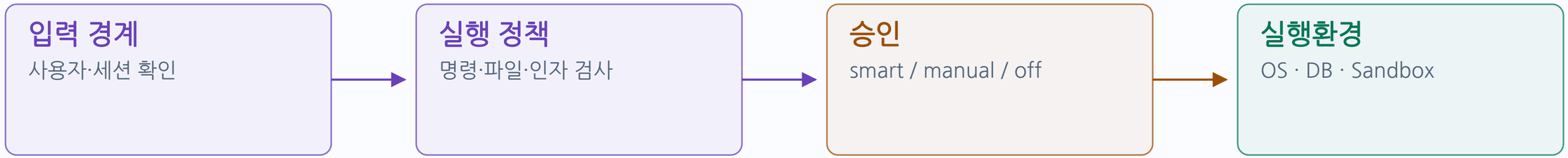


공유 파일·DB 권한은 별도로 격리해야 한다

Subagent의 결과를 받은 부모가 다음 작업을 이어갑니다.

실행 권한과 Hermes Demo

입력 검사, 승인, 실행환경, 복구 기능을 확인합니다.



패턴 탐지 + 실제 권한
위험 탐지와 접근 제어를 함께 사용
무인 실행의 deny 정책 별도

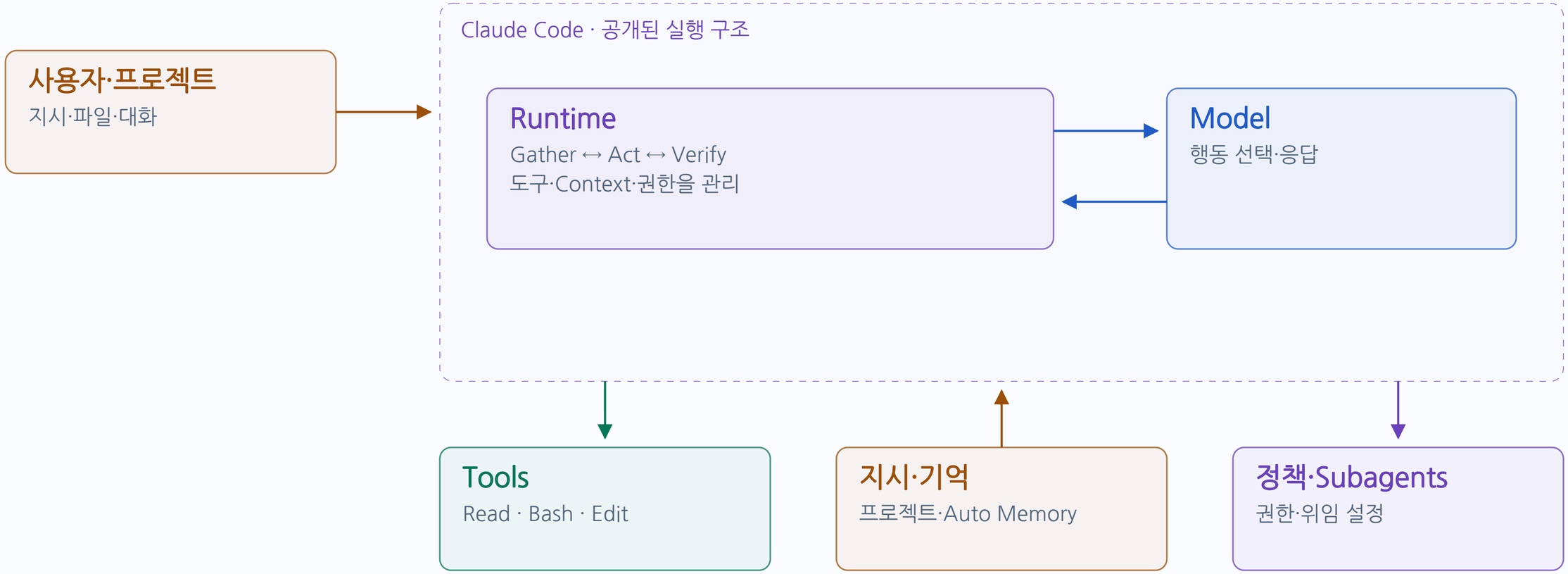
복구 Checkpoint
opt-in · 지원 범위에 따른 복구
외부 변경 전체 rollback은 아님

Demo 1 Context → Tool → 세션·기억 저장

> Demo에서는 모델 요청, Tool 결과, 저장된 상태를 살펴봅니다.

Claude Code의 실행 구조

Context를 모으고, 도구를 실행하고, 결과를 확인합니다.



도구와 지침, 권한 설정이 하나의 실행환경에 연결됩니다.

프로젝트 지침과 Auto Memory

사람이 정한 규칙과 Agent가 남긴 기억

제품	사람이 작성하는 지시	Agent가 관리하는 기억
Hermes	SOUL · 프로젝트 지침	MEMORY.md / USER.md 세션 snapshot
Claude Code	CLAUDE.md 필요시 @AGENTS.md import	Auto Memory 인덱스 필요시 세부 파일 읽기

실제 접근 권한은 도구와 실행환경에서 확인합니다.

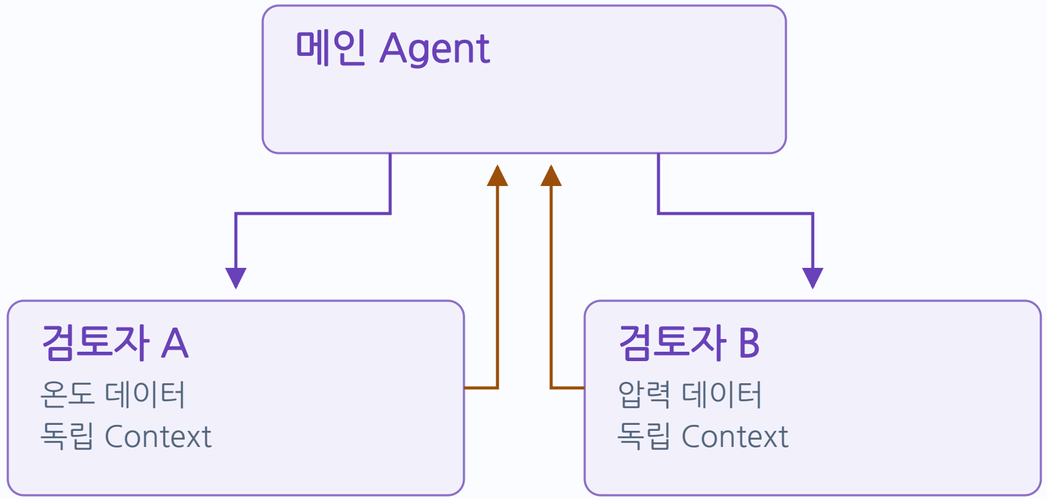
> 접근 권한은 지침 파일과 별도로 관리합니다.

Subagent와 Agent Teams

부모에게 결과를 돌려주거나, 팀원끼리 협업합니다.

Subagent

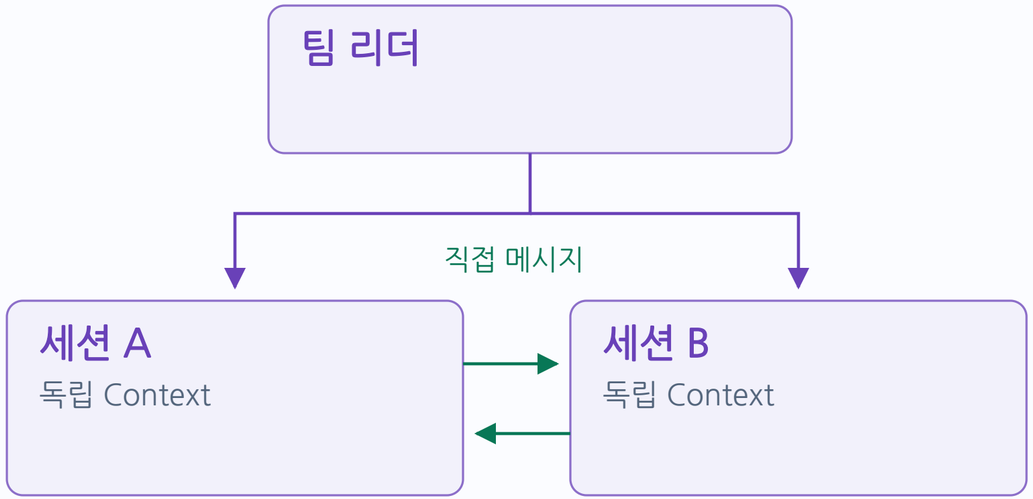
부모의 위임을 수행하고 결과를 반환



결과는 부모에게 보고

Agent Teams

독립 세션·공유 작업·직접 메시지



공유 작업 목록 · 실험적 기능 · 조정 비용

> 독립 Context를 써도 공유 파일과 자원의 충돌은 확인해야 합니다.

명령으로 세션과 기억 관리하기

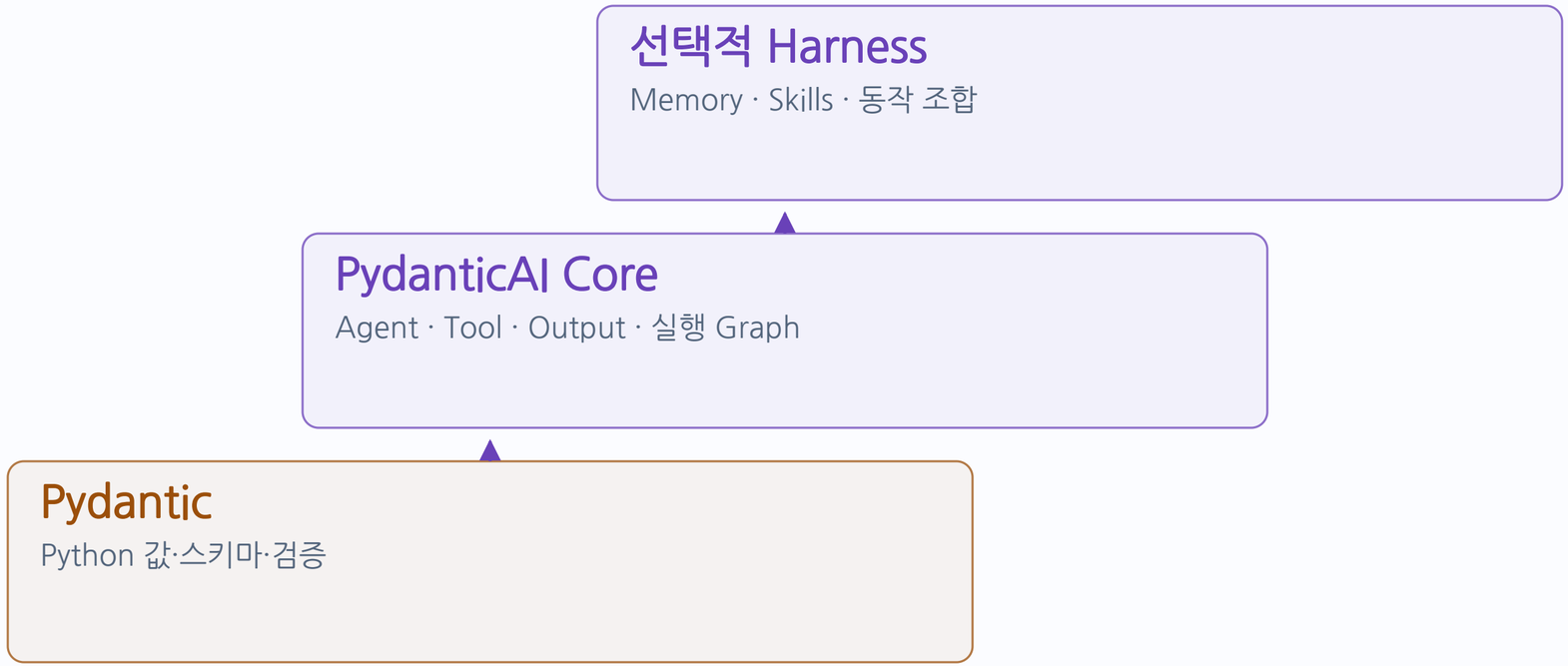
대화 재개, Prompt 갱신, 보류된 Memory 검토

제품·입구	명령	바뀌는 상태
Hermes CLI	hermes sessions refresh-prompts --session SESSION_ID	저장된 Prompt 갱신 경로
Hermes Slash	/memory pending	보류된 기억 쓰기 검토
Claude Code	/resume · /compact	대화 선택 · Context 압축

명령마다 바꾸는 상태와 적용 시점이 다릅니다.

Pydantic에서 PydanticAI로

값의 검증에서 Agent 실행과 선택적 Harness까지



함께 연결하는 도구

pydantic-graph

명시적 Graph·Workflow

Evals / Logfire

평가와 실행 관찰

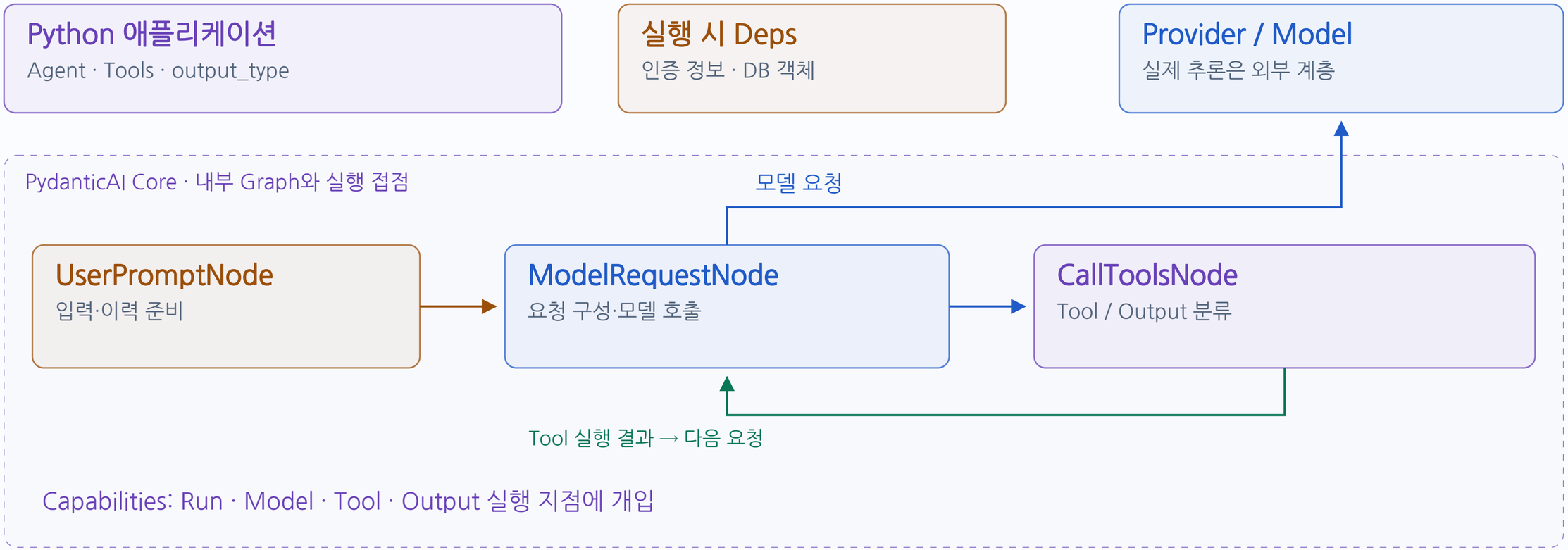
Provider / Model

모델 추론은 외부 계층

Core로 시작하고, 필요한 Harness 기능을 추가할 수 있습니다.

PydanticAI 전체 구조

개발자가 정의한 Agent를 내부 Graph가 실행합니다.



연결: History 저장소 / Tool 서비스 / Output 검증 / 선택 Harness

> 모델·Tool·저장소를 애플리케이션에 맞게 연결합니다.

Agent와 Tool을 코드로 정의하기

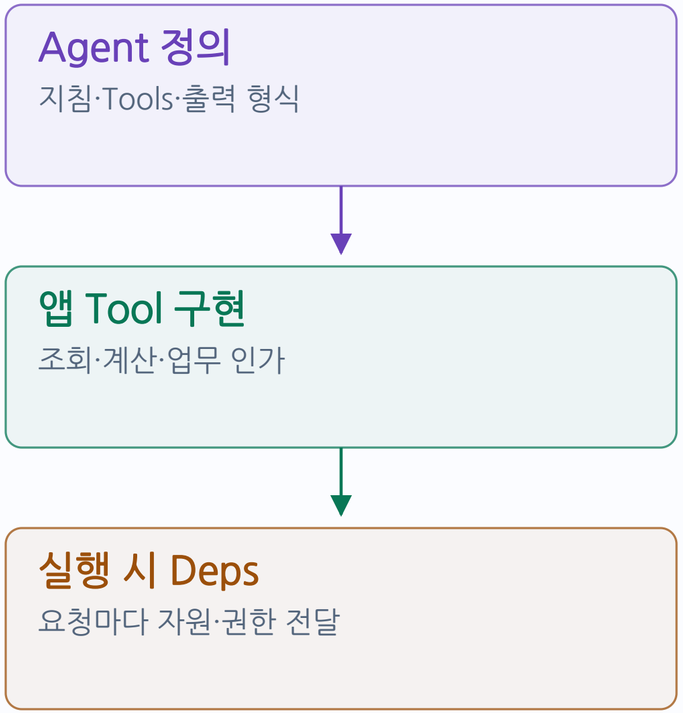
지침, 실행 함수, 의존성을 연결하는 기본 코드

실제 API 축약 · 앱 정의/연결 설정은 노트 참고

```
agent = Agent(model, deps_type=DemoDeps,
instructions='도구 근거로만 답하라.')

@agent.tool
async def get_bin_summary(
ctx: RunContext[DemoDeps], lot_id: str
) -> dict:
return await ctx.deps.query(lot_id)

result = await agent.run(
'LOT-DEMO의 BIN-X를 조회해줘', deps=deps)
```

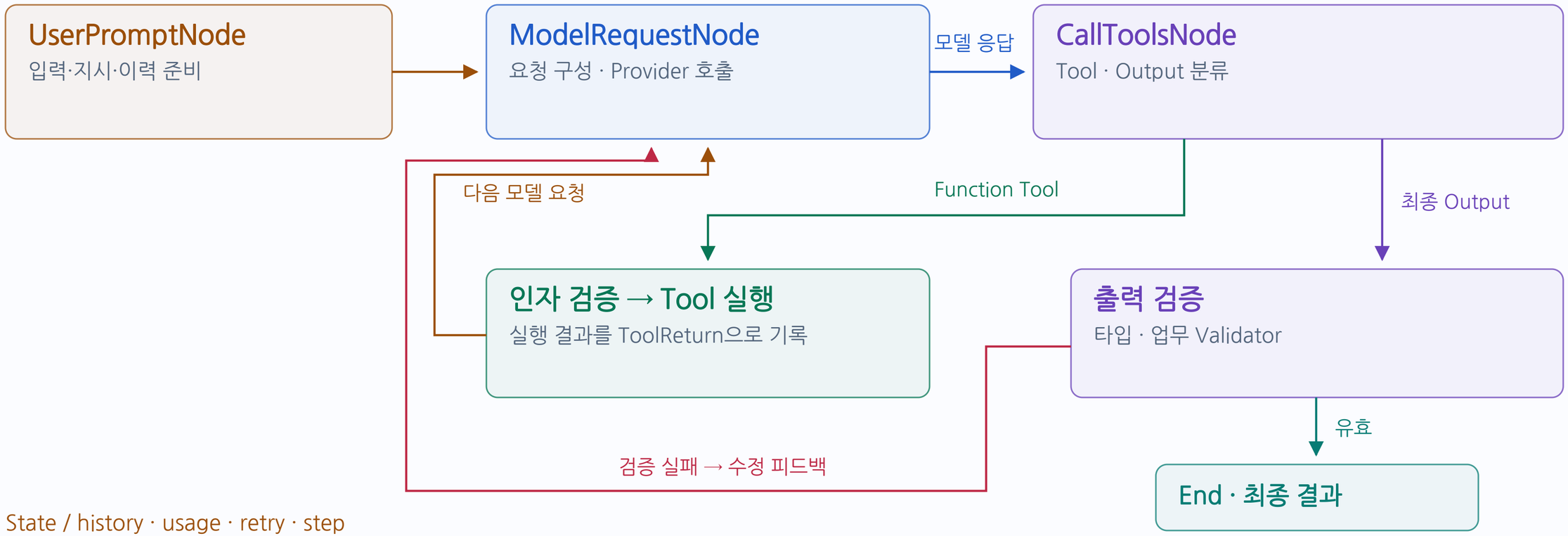


업무 데이터 조회와 권한 확인은 애플리케이션에서 구현합니다.

세 노드로 보는 Agent 실행

입력 준비, 모델 요청, Tool과 최종 출력 처리

PydanticAI 내부 실행 Graph · pydantic-graph

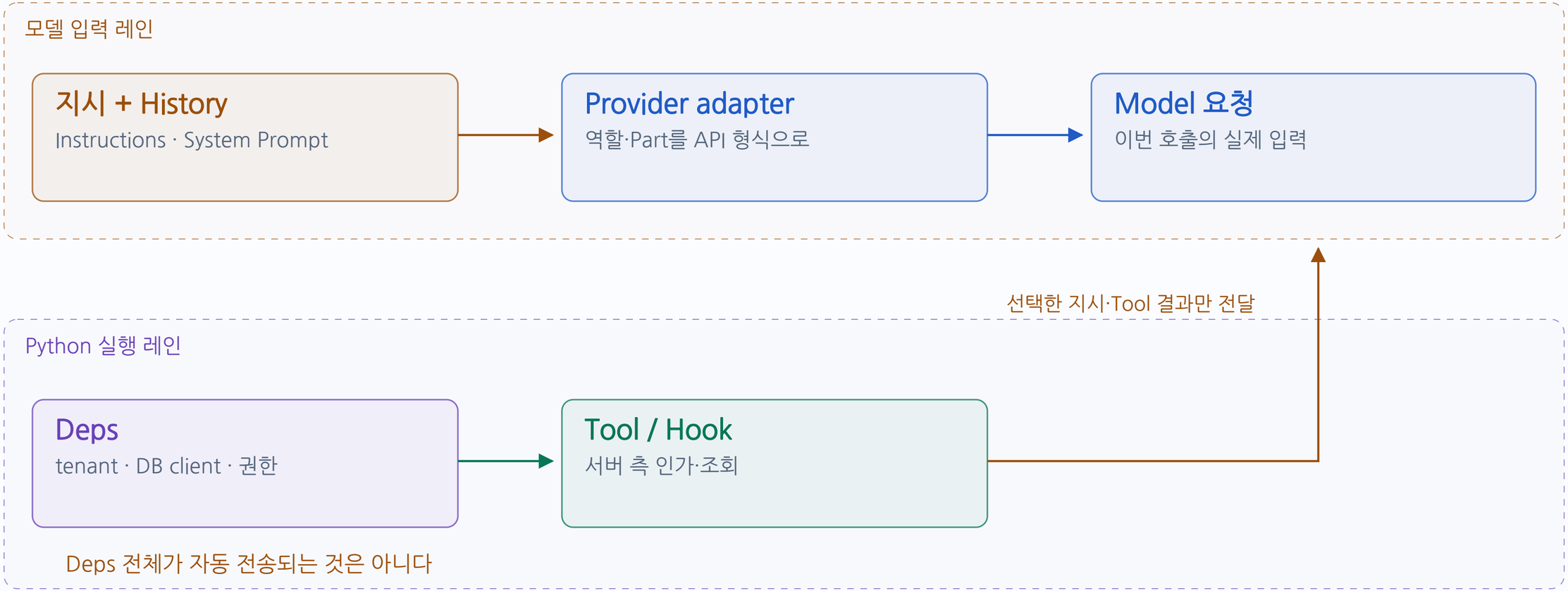


State / history · usage · retry · step

Tool 결과와 수정 피드백은 다음 모델 요청으로 이어집니다.

모델 입력과 Python 실행 자원

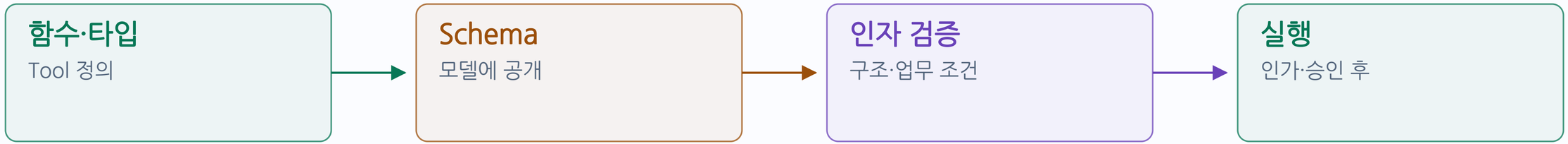
지시·History와 Deps가 전달되는 경로



Deps에서 사용한 정보도 Tool 결과로 반환하면 모델에 전달됩니다.

함수 정의에서 Tool 실행까지

스키마를 공개하고, 인자를 검증한 뒤 실행합니다.



등록 방식	Python 실행 문맥	사용 예
@agent.tool	RunContext를 받음	tenant·DB·권한이 필요한 조회
@agent.tool_plain	일반 인자	독립 계산·변환

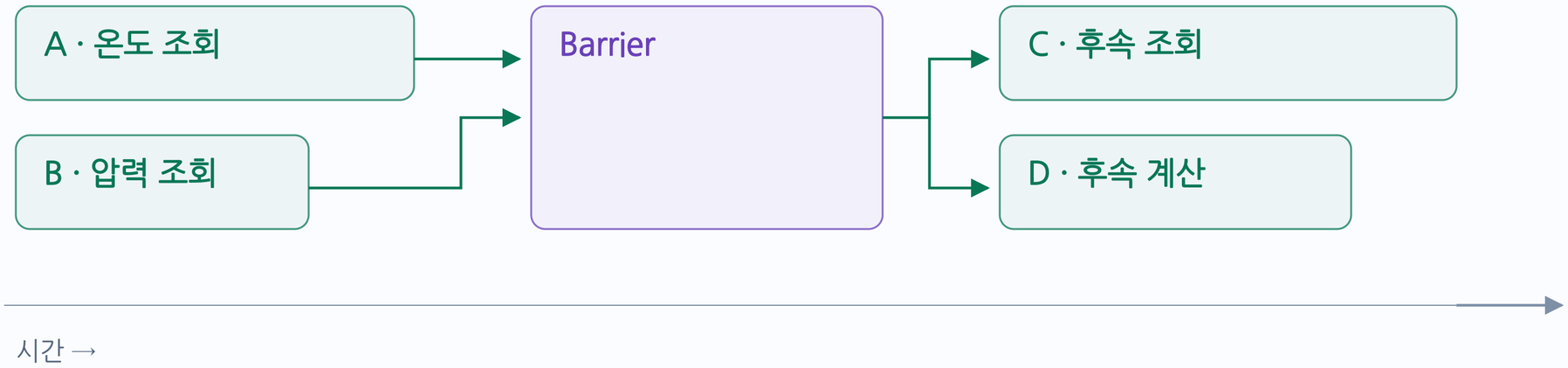
> 모델에 도구를 보여주는 것과 실행을 허가하는 것은 별개입니다.

함께 실행할 작업과 기다릴 작업

독립 작업은 병렬로, 순서가 필요한 작업은 barrier로 처리합니다.

모델이 제안한 순서 A · B → sequential Tool → C · D

실제 실행



async Tool: task 동시 스케줄

sync Tool: worker thread

sequential=True는 실행 순서를 정합니다. 데이터 snapshot은 별도로 관리합니다.

> 여러 조회 결과를 합칠 때는 대상과 기준 시점도 맞춰야 합니다.

출력 형식과 업무 정확성 검증

타입 검사 이후에도 계산과 근거를 확인합니다.

ToolOutput

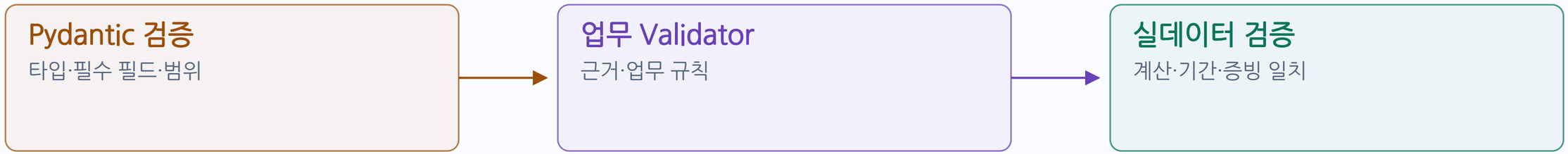
출력용 Tool Schema 제공
모델이 구조화 호출 생성

NativeOutput

Provider의 구조화 기능
모델·스키마 지원 확인

PromptedOutput

형식 지시 + 파싱·검증
형식 위반은 수정 경로



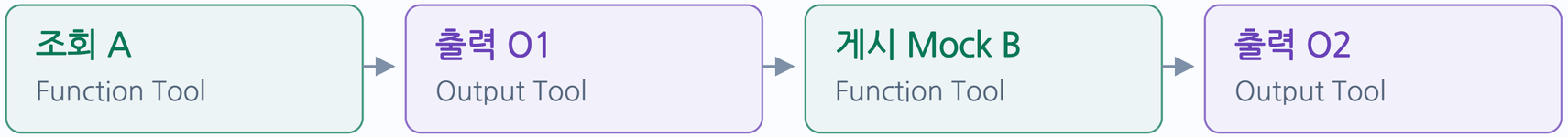
output_type은 최종 출력 형식 · Tool return type은 개별 도구의 반환 타입

> 형식이 맞는 답변도 내용은 틀릴 수 있습니다.

EndStrategy에 따른 Tool 실행

같은 응답을 받아도 종료 정책에 따라 실행 범위가 달라집니다.

모델이 한 응답에서 제안한 순서



전제: O1·O2 모두 유효 / Retry 없음 / 비스트리밍 / barrier 없음

early O1만 처리하고 종료	생략	O1 선택	생략	생략
graceful 뒤 Function도 실행	실행	O1 선택	실행	생략
exhaustive 모두 실행 · 기본 병렬	실행	O1 선택	실행	실행 · 미선택

최종 결과는 모델이 제안한 순서에서 첫 유효 Output을 선택합니다.

> graceful에서는 최종 출력 뒤의 Function Tool도 실행될 수 있습니다.

무엇을 다시 시도할까?

인자·출력 수정, API 장애 복구, 업무 판단 오류

발생 위치	되돌리는 방식	별도 설계
인자·출력 오류	검증 실패 → 모델 수정 피드백	ModelRetry · 재시도 한도
Provider/API 장애	429·5xx·연결 실패 → 통신 복구	백오프 · timeout · 중첩 retry
업무 판단 실패	유효한 형식이어도 분석은 틀릴 수 있음	정답·평가·사람 검토

UsageLimits + 시간 제한 + Tool 부작용 중복 방지는 별도 관리

> 재시도가 여러 계층에 있으면 실제 요청 횟수가 늘어날 수 있습니다.

Hook과 Capability 구성

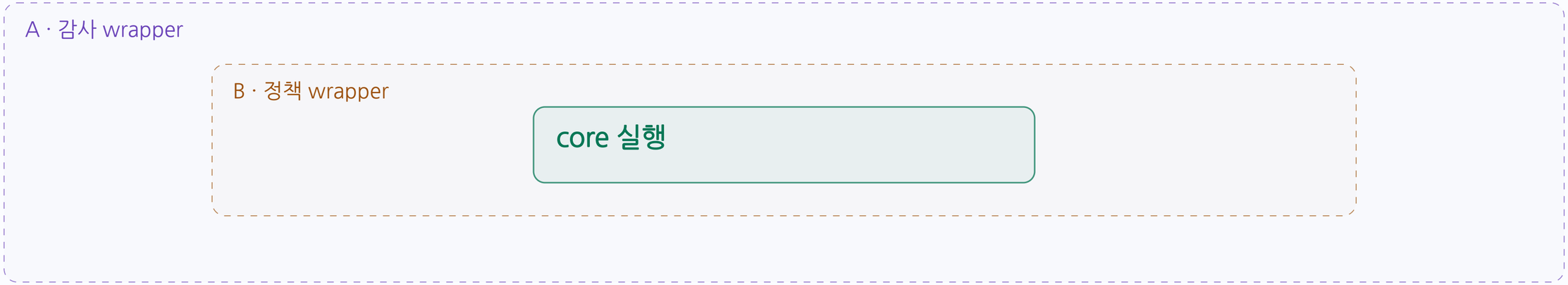
실행 시점에 개입하고, 관련 기능을 묶어 재사용합니다.

Hook

모델 요청·Tool·Run의 실행 지점

Capability

지시·도구·Hook·설정을 묶는 기능



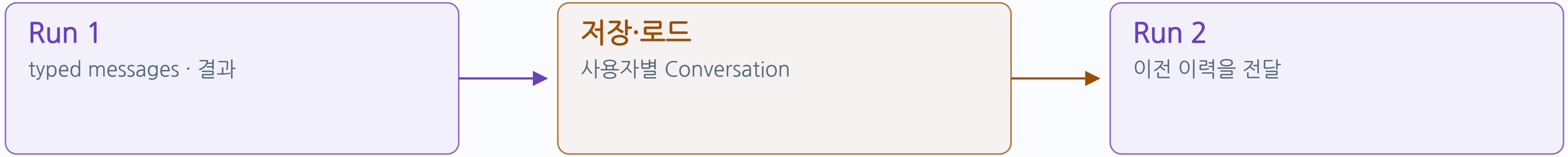
A.enter → B.enter → core → B.exit → A.exit

handler를 생략하는 wrapper는 안쪽 정책 Hook도 건너뛸 수 있다.

> wrapper 순서에 따라 안쪽 Hook의 실행 여부가 달라집니다.

이전 대화를 다음 Run으로 이어가기

이력을 저장하고 필요한 대화에 다시 전달합니다.



실제 API 축약 · 앱 정의/연결 설정은 노트 참고

```
first = await agent.run('BIN-X를 조회해줘')
second = await agent.run(
'분모는?', message_history=first.all_messages())
```

함께 이어갈 상태

history · usage · ID
현행 conversation API

대화 저장, 사용자 구분, 누적 사용량을 함께 관리합니다.

Context 압축과 원본 보관

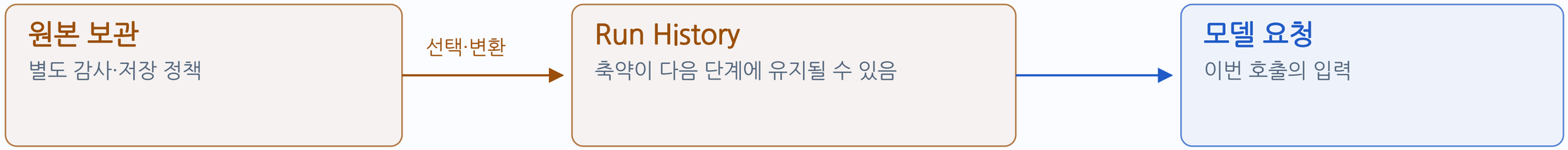
Core와 Harness가 제공하는 압축 방법

Core

Provider-native compaction
지원 모델·API 조건 필요

별도 Harness

clear · sliding window · summarize
모델 비종속 전략 선택

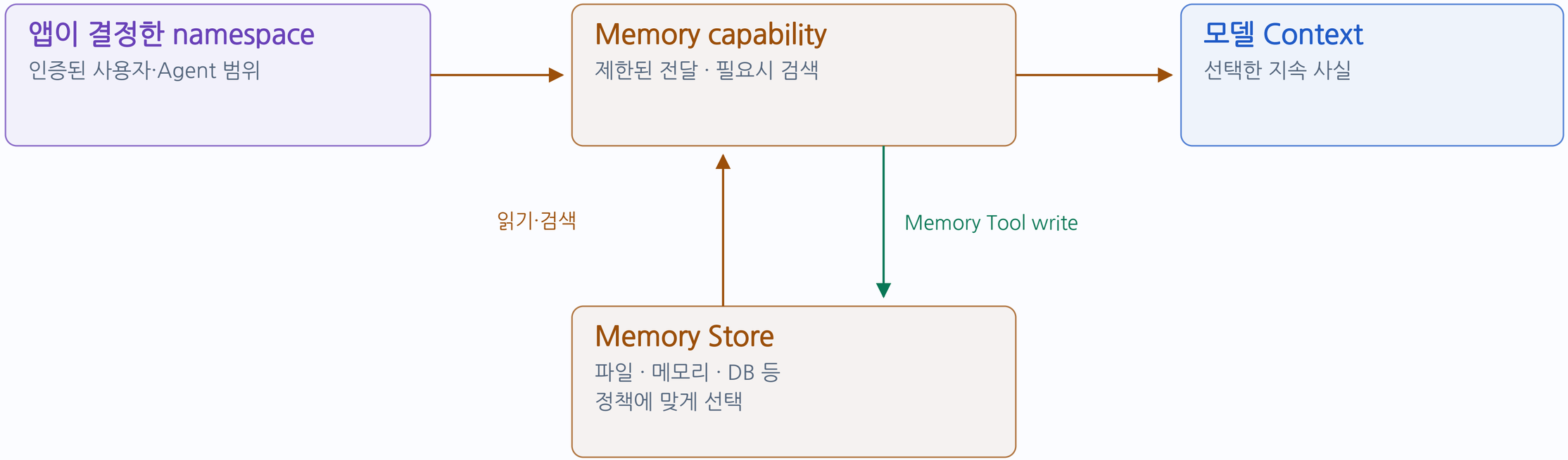


호출 쌍과 수치·근거를 보존하고, 원본은 별도로 보관합니다.

> 모델에 보낼 이력과 감사용 원본은 따로 관리합니다.

Harness의 Memory 기능

대화가 끝난 뒤에도 필요한 사실을 저장하고 불러옵니다.



namespace로 저장 범위를 나누고, 인증과 DB 권한은 따로 확인합니다.

> 저장 범위뿐 아니라 읽기·수정 권한과 유효기간도 정합니다.

Skill 로딩과 MCP 연결

절차 문서를 읽는 경로와 외부 도구를 호출하는 경로

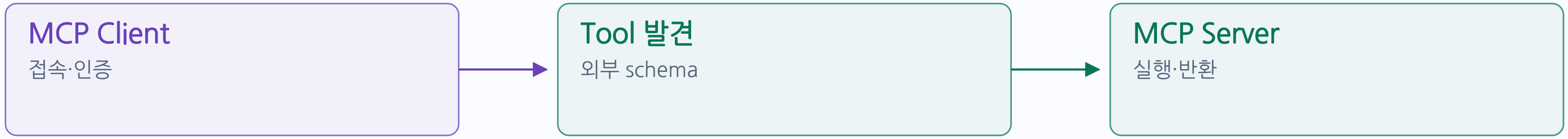
Skills

문서형 절차를 모델 입력에 제공



MCP

외부 기능의 발견·호출을 연결



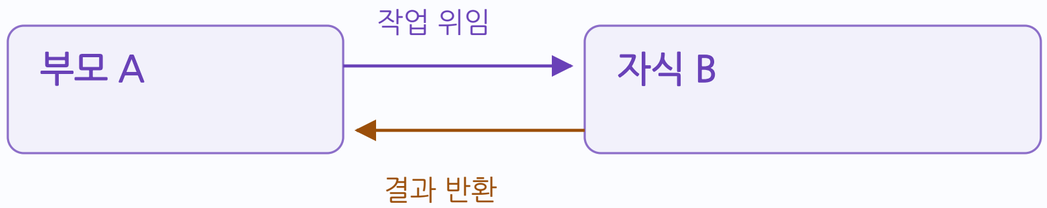
> 사용할 절차와 서버, 인증 설정을 준비해야 합니다.

Agent 사이의 작업 전달

위임, 라우팅, Graph, Advisor의 실행 방식

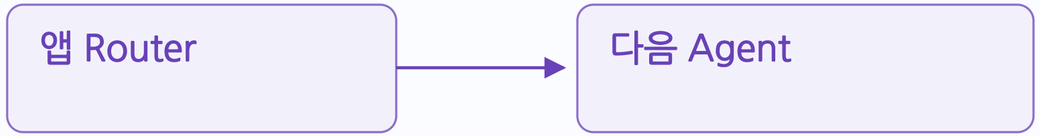
도구 내부 위임

부모 → tool(B.run) → 부모



Programmatic Handoff

앱 코드가 다음 Agent를 선택



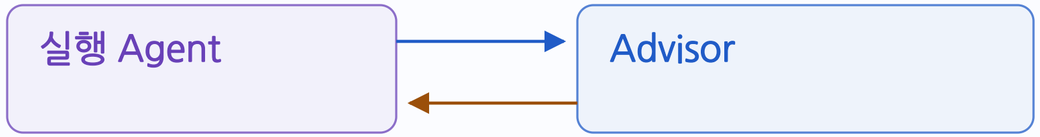
Graph Workflow

노드·상태·분기를 명시



Harness Subagents / Advisor

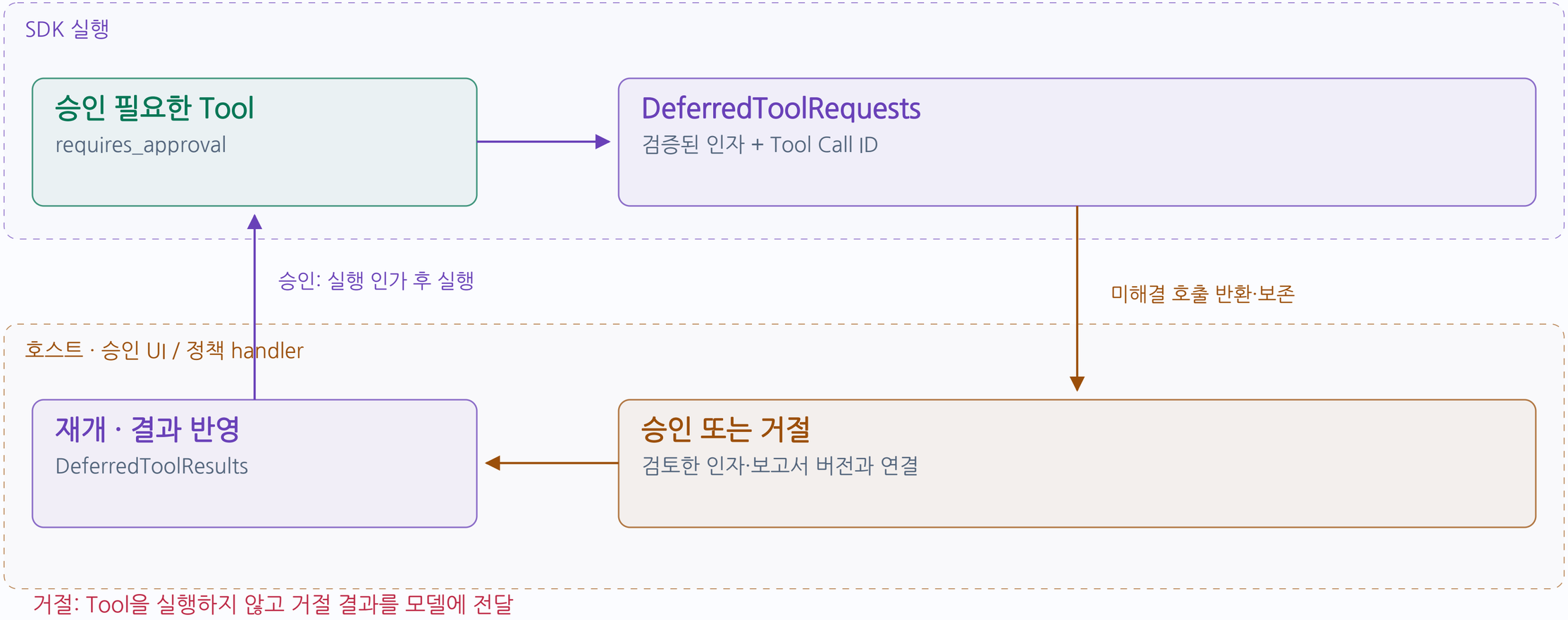
하위 작업과 조언을 역할별로 구성



누가 다음 작업을 정하고 어디로 결과를 돌려주는지 확인합니다.

승인을 기다렸다가 실행 재개하기

보류된 호출을 검토하고, 승인 결과를 전달합니다.



검토한 호출과 인자를 실행 시점에 다시 확인합니다.

Core와 Harness로 구성하는 Agent

실행 기능을 조합하고 업무에 필요한 코드를 연결합니다.

책임	Core	선택 Harness	업무 애플리케이션
실행	Loop · Tools · Output	동작 스택·위임	업무 함수·종료 기준
정보	메시지·이력 API	Memory · Skills	데이터 형식·보관·삭제
통제	검증·Hook·승인 경로	정책 조합	인증·인가·감사·운영

Logfire / OTel은 실행 관찰, Evals는 목표 기준 평가에 연결한다.

> 데이터, 권한, 결과 검증은 업무에 맞게 설계합니다.

PydanticAI Demo

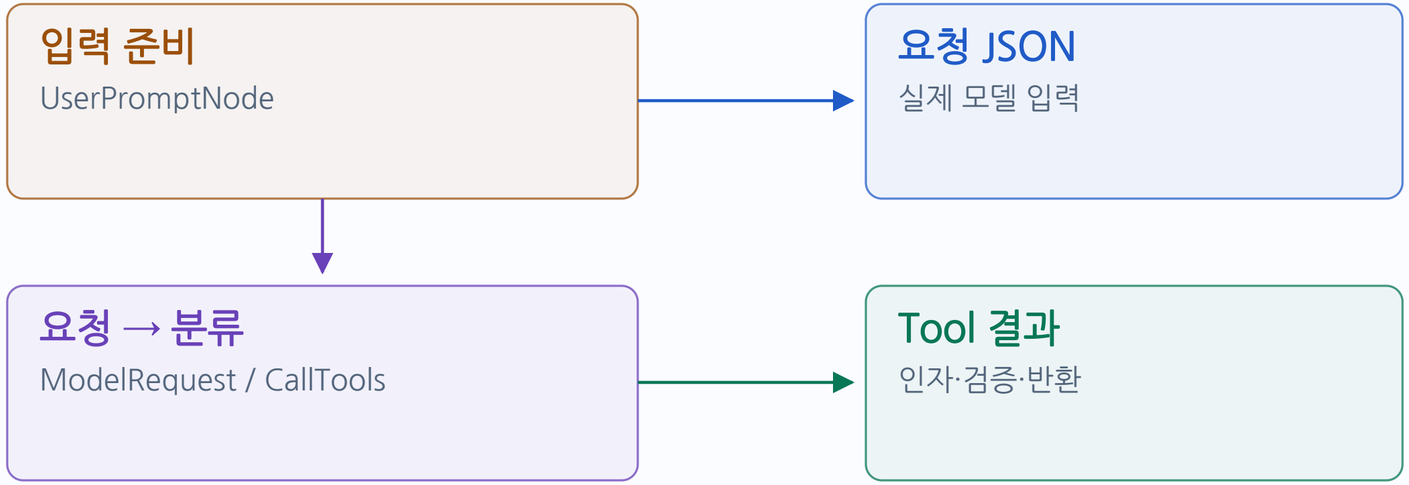
정의 코드에서 모델 요청과 Tool 결과까지 따라갑니다.

개발자 코드

```
Agent(...)

@agent.tool
...

agent.run(...)
```



Demo 2 Core 실행 → Harness 기능 추가

> Core 실행을 먼저 확인한 뒤 Harness 기능을 추가합니다.

Agent를 어디에서 실행할까?

중앙 서버와 사용자 PC에 배치했을 때의 차이

중앙 Agent 실행

공통 운영 + 사용자·작업 격리

Client → API / Queue

인증·tenant·세션 순서

Agent Workers

Tool / DB · 작업 수명·예산

사용자 로컬 실행

설치·업데이트 + 로컬 실행 권한

Desktop Agent

서명된 배포·업데이트·정책 버전

로컬 Tools / 저장소

사용자 파일·실행 계정 권한

모델 서비스 · 부하·접근 제어

Agent 실행 자원과 모델 서버 자원을 나누어 살펴봅니다.

실행 기록과 업무 결과 검증

무엇을 실행했는지, 요구한 결과가 맞는지 확인합니다.

Trace · 무엇이 실행되었나

관찰 가능한 실행 증거

실행 기록

Run / Request / Call ID
실제 인자·Tool 결과
승인·재개·오류 경로
사용한 데이터 snapshot

Evaluation · 옳게 수행했나

업무 목표와 제약 기준

업무 검증

분자·분모·기간 일치
근거·SOP revision 확인
승인 인자와 실행 일치
예측 시간 분할·누수 확인

같은 Tool도 데이터와 기준 시점이 다르면 다른 결과를 냅니다.

> 데이터 범위, 계산, 근거, 승인 내용을 함께 확인합니다.

구조를 이해하고 실제 개발로

Hermes와 PydanticAI에서 살펴본 내용

01 **Hermes**

상태·기억·도구·정책의 동작

02 **PydanticAI**

코드 정의부터 실행과 검증까지

03 **업무 적용**

데이터·권한·결과 검증

> 3차에서는 Agent를 개발하고 검증 결과와 시행착오를 공유합니다.